

Forecasting at Scale: Transforming SuperKart Historical Sales Data into Predictive Intelligence

SuperKart Forecast & Model Deployment

Austin University – Post Graduate Program in Artificial Intelligence

2026-06-08

Contents / Agenda

- Executive Summary
- Business Problem Overview and Solution Approach
- EDA Results
- Data Preprocessing
- Model Building
- Model Performance Summary
- Deployment
- Recommendation – Adding Product Price (MRP)

Executive Summary

The Challenge

To optimize inventory management and refine regional tactics, SuperKart required a scalable, data-driven solution to forecast product-level sales revenue. Utilizing a robust dataset of 8,763 product-store records across 4 distinct outlets, the objective was to pinpoint the exact structural and commercial drivers of supermarket revenue.

The Solution

We evaluated multiple advanced ensemble frameworks, including Random Forest and XGBoost, utilizing hyperparameter tuning (GridSearchCV). The models demonstrated strong alignment on test data, yielding a Test R^2 of ~ 0.66 , an RMSE of $\sim ₹616$, and a MAPE of 18.7%. Based on its superior stability and consistent error minimization, the tuned Random Forest was selected as the final production model.

Strategic Insights

Revenue is heavily dictated by store format, city tier, and pricing. Departmental Stores and Tier-1 City locations significantly outperformed Food Marts and Tier-3 settings. Notably, outlet OUT004 emerged as a critical driver, generating ₹15.4M—outperforming the other three outlets combined.

The Recommendation

Deploy the finalized Random Forest pipeline via a decoupled Flask API and Streamlit UI to immediately improve supply-chain efficiency and inventory planning. For phase-two development, integrating Product_MRP directly into production is highly recommended, as testing proves it will significantly maximize future predictive accuracy.

	Random Forest	Random Forest Tuned	XGBoost	XGBoost Tuned
RMSE	615.907	615.907	615.933	616.087
MAE	485.311	485.311	485.430	485.835
R-squared	0.669	0.669	0.668	0.668
Adj. R-squared	0.667	0.667	0.667	0.667
MAPE	0.187	0.187	0.187	0.187

Business Problem Overview and Solution Approach

Business Problem

SuperKart needs accurate **product-store level sales forecasts** to support better inventory planning, supply-chain optimization, and store performance benchmarking.

Because the target variable, **Product_Store_Sales_Total**, is continuous, this is framed as a **regression problem**.

Solution Approach / Methodology

The solution followed a structured machine-learning workflow

Exploratory Data Analysis

Identified key sales drivers, data patterns, and potential data quality issues.

Data Preprocessing & Feature Engineering

Prepared the dataset for modeling, including categorical encoding and pipeline creation.

Model Training & Comparison

Trained and compared multiple tree-based and ensemble regression models using **R^2 , RMSE, MAE, and MAPE**.

Model Tuning & Selection

Tuned the strongest candidate models and selected the best-performing model based on test performance and generalization.

Deployment

Deployed the final model as a forecasting service using an **API and web application** for business users.

EDA Results

The sales target showed an average product-store sale of **\$3,464**, with values ranging from **\$33 to \$8,000** and a roughly symmetric distribution.

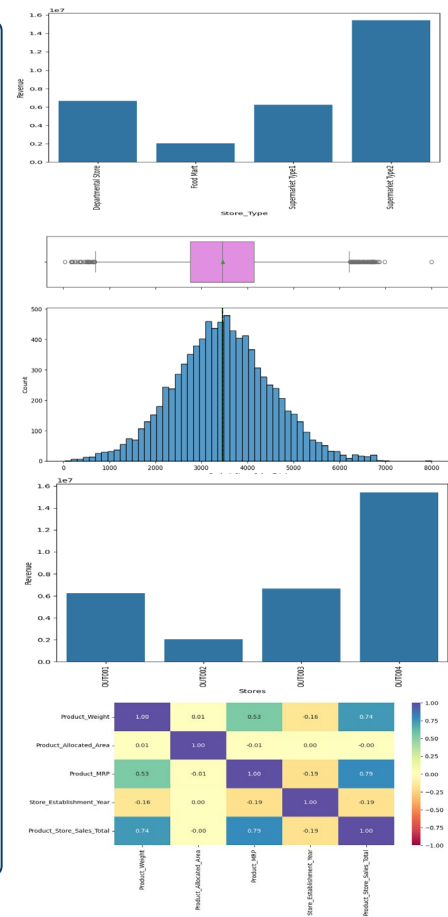
The strongest sales drivers were **Product_MRP** and **Product_Weight**, with correlations of **0.79** and **0.74**, respectively. In contrast, **Product_Allocated_Area** showed almost no linear relationship with sales.

Sales performance varied significantly by store format. **Departmental Stores** generated the highest average sales, followed by **Supermarket Type 1**, **Supermarket Type 2**, and **Food Mart**.

City tier and store size also showed clear differences in sales. **Tier 1 cities** and **high-size stores** generated the strongest sales performance, while **Tier 3 cities** and smaller stores underperformed.

Outlet-level revenue was highly concentrated, with **OUT004 contributing \$15.4M**, followed by **OUT003 at \$6.7M**, **OUT001 at \$6.2M**, and **OUT002 at \$2.0M**.

Business implication: pricing, product attributes, store format, and location are the strongest indicators of expected sales and should guide inventory planning and regional sales strategy.



Data Preprocessing

Future Engineering

New variables were created to improve model performance and capture business-relevant patterns:

Store_Age_Years

Calculated store age using: $2025 - \text{Store_Establishment_Year}$

Product_Id_char

Extracted the first two characters of Product_Id to identify broader product groups such as FD, DR, and NC

Product Category Grouping

Consolidated 16 detailed product types into two broader groups: Perishables and Non-Perishables

Data Preprocessing

Data Cleaning

The dataset was reviewed and standardized before modeling:

Standardized inconsistent values in Product_Sugar_Content, changing "reg" to "Regular."

Confirmed there were no missing values and no duplicate rows in the dataset

Data Preparation for Modeling

The final modeling dataset was prepared using a reusable preprocessing pipeline:

Dropped identifier and redundant columns:

Product_Id, Product_Type, Store_Id, and Store_Establishment_Year

Applied **OneHotEncoder** to categorical variables with handle_unknown="ignore" and passed numerical variables through unchanged

Split the data into a 70/30 train-test split (6,134 train / 2,629 test), random_state = 1

Data Preprocessing

Future Engineering

New variables were created to improve model performance and capture business-relevant patterns:

Store_Age_Years

Calculated store age using: $2025 - \text{Store_Establishment_Year}$

Product_Id_char

Extracted the first two characters of Product_Id to identify broader product groups such as FD, DR, and NC

Product Category Grouping

Consolidated 16 detailed product types into two broader groups: Perishables and Non-Perishables

Data Background and Contents

Dataset Overview

The dataset contains 8,763 rows and 12 columns, with no missing values and no duplicate records.

Product Attributes

Product-level features include:

- Product_Weight
- Product_Sugar_Content
- Product_Allocated_Area
- Product_Type
- Product_MRP

Data Background and Contents

Store Attributes

Store-level features include:

- Store_Establishment_Year
- Store_Size
- Store_Location_City_Type
- Store_Type

Identifiers

The dataset includes product and store identifiers:

- Product_Id
- Store_Id
- 4 outlets: OUT001 to OUT004

Target Variable

- Product_Store_Sales_Total

This represents the total sales revenue of a product at a specific store.

Data Coverage

The dataset covers 16 product types across 4 stores, spanning 3 city tiers and 3 store sizes.

Model Building

- Built Random Forest and XGBoost regressors inside one preprocessing pipeline (one-hot encoded store/product categoricals); evaluated on the held-out test set.
- Both base models perform almost identically — Test $R^2 \approx 0.668$, RMSE ≈ 616 , MAPE $\approx 18.7\%$ — a statistical tie, with Random Forest holding a slight, consistent edge.
- R^2 and RMSE are two views of the same errors ($R^2 = 1 - \text{RMSE}^2/\text{Var}(Y)$), so they rank the models the same; MAPE $\approx 18.7\%$ agrees, confirming the tie.
- The models explain $\sim 67\%$ of sales variance, giving a solid baseline for demand planning.

Model (base)	Test R^2	RMSE (₹)	MAE (₹)	MAPE
Random Forest	0.668	616	485	18.7%
XGBoost	0.668	616	485	18.7%

Model Improvement - Hyperparameter Tuning

Tuning Approach

Both finalist models were tuned using GridSearchCV.

- Cross-validation: **3-fold CV**
- Scoring metric: **R^2**

Random Forest Tuning Results

Best parameters:

- `max_features = "sqrt"`
- `n_estimators = 100`
- `max_depth = None`

The selected parameters were **close to the default model** settings.

Test performance:

- Test R^2 remained unchanged at 0.668
- Tuned Random Forest performed the same as the base Random Forest model

XGBoost Tuning Results

Best parameters:

- `gamma = 0`
- `n_estimators = 100`

The selected parameters were also near the default settings

Test performance:

- Test R^2 remained unchanged at 0.668
- RMSE remained around 616
- No meaningful performance gain was observed

Key Takeaway

Hyperparameter tuning did not produce a measurable improvement.

The models appear to have reached the performance ceiling based on the available features.

Since Random Forest and XGBoost remained statistically tied, the tuned Random Forest model was selected as the final model due to its stability, consistency, and simpler deployment profile.

Model Performance Summary

- Final selected model: Tuned Random Forest (serialized with joblib and deployed).
- All four models are statistically tied — Test $R^2 \approx 0.668$, differing only at the 4th decimal — but Random Forest holds a slight, consistent edge over XGBoost across every metric.
- Random Forest (tuned) was selected for deployment; the price-inclusive analysis (next slide) reinforces Random Forest as the stronger model family.
- Selected-model test performance: $R^2 = 0.668$, RMSE = 616, MAE = 485, MAPE = 18.7%.

Model	Test R^2	Adj R^2	RMSE	MAE	MAPE
Random Forest (tuned) ★ deployed	0.668	0.667	616	485	18.7%
Random Forest (base)	0.668	0.667	616	485	18.7%
XGBoost (base)	0.668	0.667	616	485	18.7%
XGBoost (tuned)	0.668	0.667	616	486	18.7%

Deployment

Production Model

The tuned Random Forest model was serialized using joblib and deployed as the production forecasting model.

Backend API

The model is served through a Flask REST API.

- Endpoint: POST /v1/predict
- Input: product and store attributes
- Output: predicted sales value returned in JSON format

Frontend Application

A Streamlit web application was created to make the model accessible to business users.

The app supports:

- Single product-store sales predictions
- Batch scoring using CSV uploads

Hosting and Containerization

Both the backend API and frontend Streamlit app were containerized using Docker.

The solution is hosted on Hugging Face Spaces for deployment and accessibility.

Deployment Links

• Backend API:

<https://araujohn-rkt-superkart-backend.hf.space>

Endpoint:

POST /v1/predict

• Frontend Streamlit App:

<https://araujohn-rkt-superkart-app.hf.space>

Recommendation: Adding Product Price (MRP)

- Product_MRP (price) is the single strongest predictor of sales in EDA (correlation 0.79), yet the current production model excludes it per project scope.
- A controlled experiment adding Product_MRP improves every metric dramatically — and confirms Random Forest as the best model, widening its edge over XGBoost.
- Business impact: typical forecast error roughly halves (RMSE 616 -> 291), enabling materially tighter inventory and supply-chain planning.
- Recommendation: incorporate Product_MRP (and Product_Weight) in the next model iteration once approved for use.

Feature set	Best model	Test R ²	RMSE	MAPE
Without price (current)	Random Forest (tuned)	0.668	616	18.7%
With price (proposed)	Random Forest (tuned)	0.926	291	5.2%

Key Takeaways & Business Recommendations

Key Business Drivers

Pricing and store format are the strongest sales levers in the dataset.

- Product_MRP has a strong correlation with sales at 0.79
- Departmental and Tier-1 stores generate approximately 3x higher sales than Food Mart and Tier-3 stores

Replicate High-Performing Store Patterns

SuperKart should analyze and replicate the success factors behind OUT004 and Departmental / Tier-1 store formats.

Focus areas include:

- Product assortment
- Pricing strategy
- Inventory availability
- Store format and location strategy

Model Selection

Random Forest and XGBoost performed very similarly, with Test R^2 around 0.668.

The tuned Random Forest model was selected and deployed due to its slight performance edge, stability, and simpler production profile.

XGBoost can be selected for a large amount of data, its parallelism improves performance.

Highest-Impact Model Improvement

The next major improvement should be to add **Product_MRP** to the model.

A controlled test showed that including **Product_MRP** could:

- Increase Test R^2 from approximately 0.67 to 0.93
- Reduce typical prediction error by roughly half

Production Readiness

The forecasting solution is production-ready through a **Flask API** and Streamlit web app hosted on Hugging Face Spaces.

Recommended operating approach:

- Monitor prediction accuracy over time
- Retrain periodically as new sales data becomes available



Happy Learning !

